

数据库辅导

Question1 : SQL语句与关系代数

SQL语句与关系代数都是查询的一种语言，涉及到的知识有SQL语句和关系代数运算的书写，查询优化问题。

SQL语句设计单关系查询、多关系查询中的自然连接、左连接、右链接等 还有聚合函数的使用，where条件、分组聚集、having子句的使用等

关系代数的表达式书写也要会

对于关系代数的树的查询优化，也是一个考点。

查询优化可以通过利用表达式的等价变换规则来实现，主要的优化策略包括

选择下移

三 查询树的基本优化策略

1. “选择下移” 优化策略

讨论3.如何利用等价规则进行关系表达式的转换?

1)“选择下移”有何优化效果?

查询：2009年有课的Music系的所有教师名字以及每个教师所教授的课程名称

$$\Pi_{name, title}(\sigma_{dept_name = \text{“Music”} \wedge year = 2009} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$$

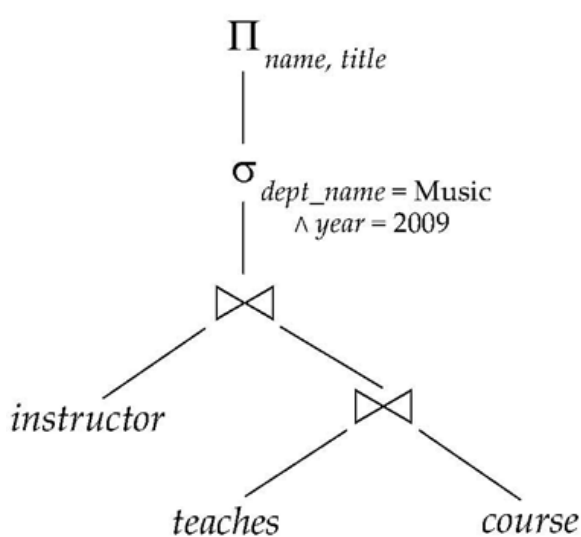
- 使用自然连接的结合律(Rule 6a):

$$\Pi_{name, title}(\sigma_{dept_name = \text{“Music”} \wedge year = 2009} ((instructor \bowtie teaches) \bowtie \Pi_{course_id, title}(course)))$$

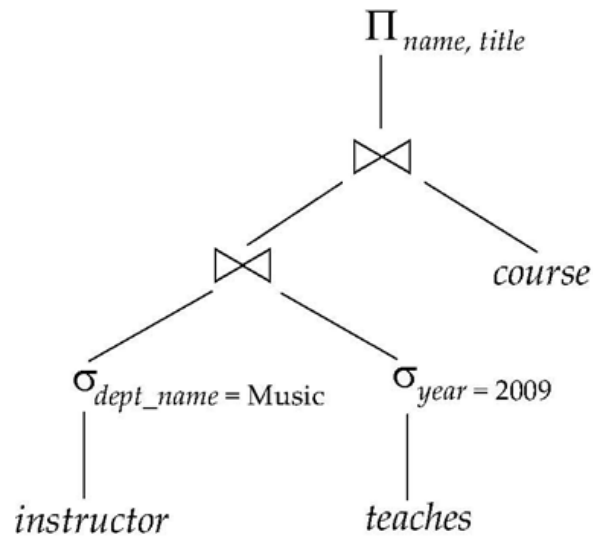
- 尽早执行选择

$$\Pi_{name, title}((\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie \sigma_{year = 2009}(teaches)) \bowtie \Pi_{course_id, title}(course))$$

- 查询树优化过程(图示法)



(a) Initial expression tree



(b) Tree after multiple transformations

投影下移

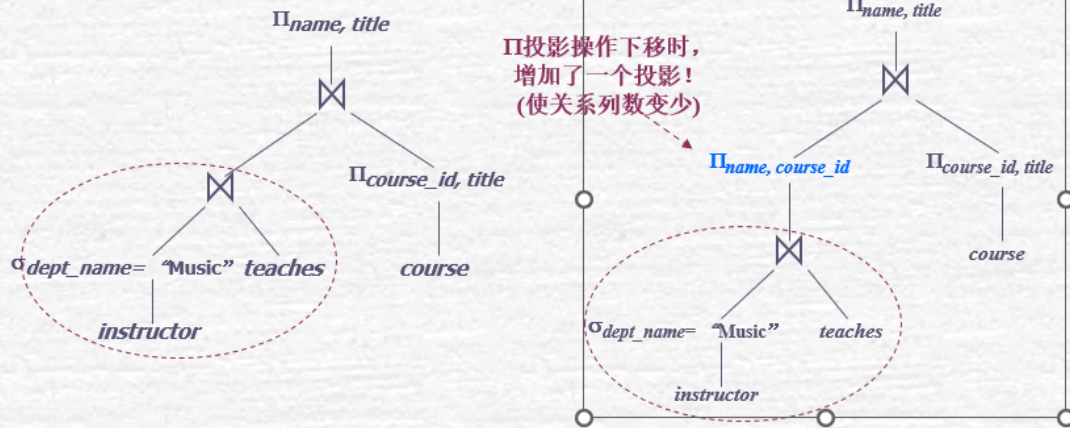
2. “投影下移” 优化策略

2) “投影下移” 有何优化效果?

- 考虑: $\Pi_{name, title}((\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$
- When we compute $\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches$ 含8个属性的大关系!
 we obtain a relation whose schema is:
 $(ID, name, dept_name, salary, course_id, sec_id, semester, year)$
- 通过使用等价规则 8. a 及 8. b 先做投影运算, 可以从模式中去掉几个属性。
- $$\Pi_{name, title}(\Pi_{name, course_id}(\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$$
含2个属性的小关系!
- 通过去除不必要的属性, 可以减少中间结果的列数, 从而减少中间结果的大小。
- 查询树优化过程(图示法)

优化前、后的查询树有何不同？

“投影下移” 查询树优化过程



选择连接

三 查询树的基本优化策略

3. “选择连接顺序” 优化策略

3) “选择连接” 次序有何优化效果？

- 考虑如下表达式：

$$\Pi_{name, title} ((\sigma_{dept_name = \text{"Music"}} (instructor) \bowtie teaches) \bowtie \Pi_{course_id, title} (course))$$

- 能够首先计算： $teaches \bowtie \Pi_{course_id, title} (course)$ ，然后在与下面表达式做连接

连接结果（记录数）更大

$$\sigma_{dept_name = \text{"Music"}} (instructor) ?$$

但是第一个连接可能得到一个很大的结果。

是先做 $teaches \bowtie course$ 好？
还是 $instructor \bowtie teaches$ 好？

- 大学老师中属于音乐学院的老师只有很少的一部分

- 因此最好先计算

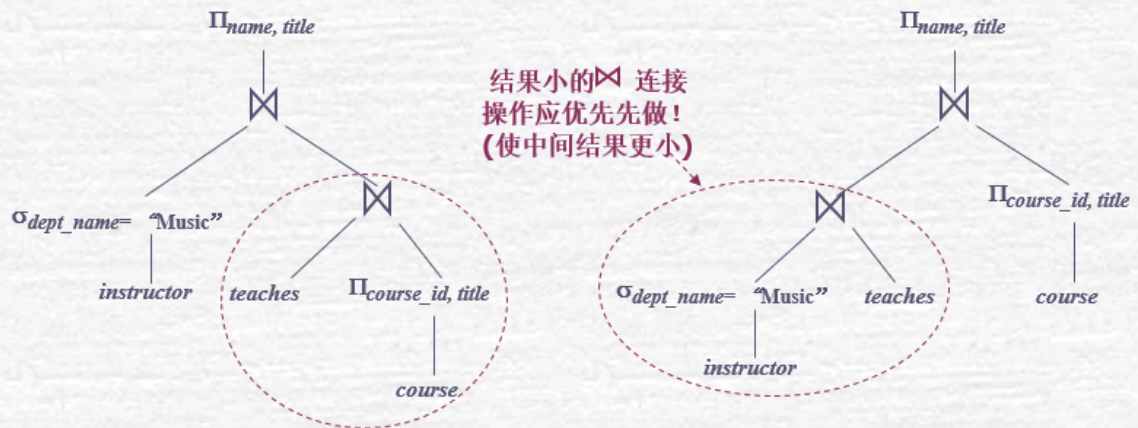
$$\sigma_{dept_name = \text{"Music"}} (instructor) \bowtie teaches$$

优化策略是：
小关系的连接应优先
(中间结果的元组数更少)

- 查询树优化过程(图示法)

优化前、后的查询树有何不同？

“选择连接顺序” 查询树优化过程



Question2 : 事务与并发

何为事务，事务就是一个访问数据项的程序执行单元。数据库必须保证事务的正常执行，在维护事务的时候应该具有以下性质：

- 原子性：要么全部执行，要么不执行
- 一致性：隔离执行事务的时候，保持数据库一致性
- 隔离性：尽管会发生并发，但每个事务感觉不到系统中其他事务在并发执行。
- 持久性：事务成功完成后，对数据库的改变应该是永久的。

(ACID property)

(1) 如何维护原子性和持久性：

维护日志，对没有成功执行的事务进行回滚

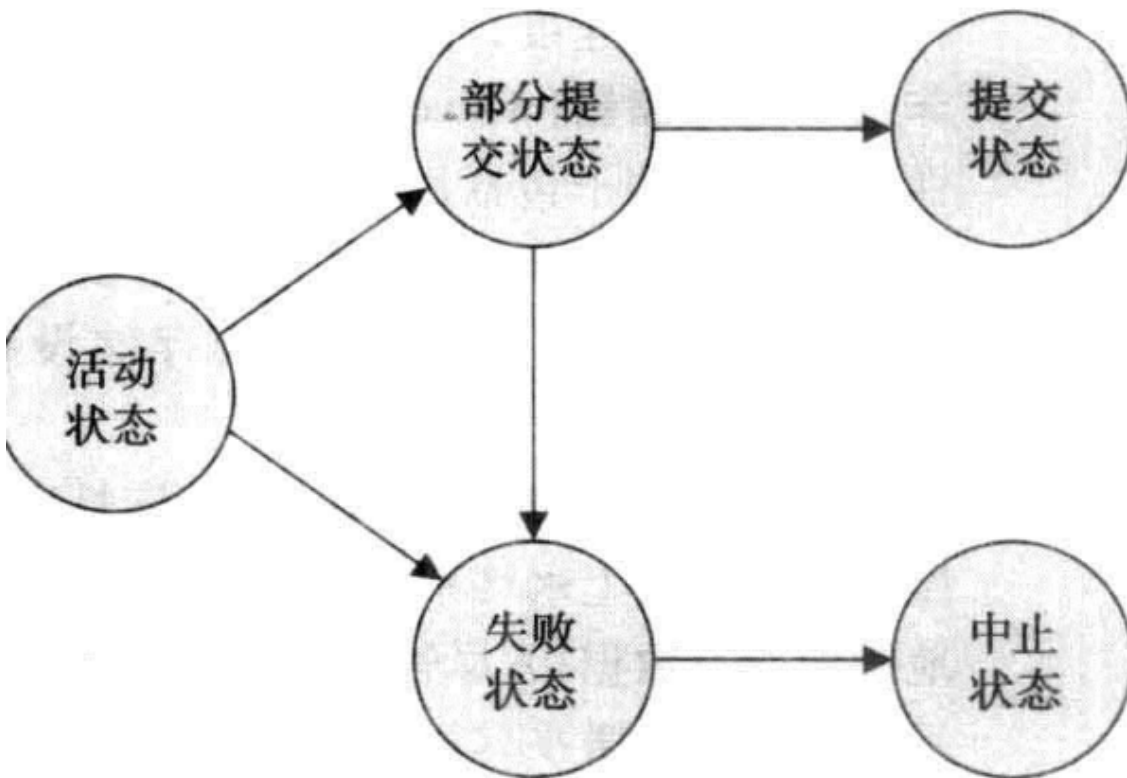


图 14-1 事务状态图

(2) 如何实现隔离性与一致性

当数据库并发执行多个事务时，调度就不再是串行，而是一个事务执行一小段时间，再执行其他事务。在调度中会出现执行结果错误的情况，这个时候就需要并发控制，把调度转化为等价的一个正常的串行调度（等价于T1 T2先后执行），也就是可串行化的调度。

具体来讲，隔离性和一致性实现的机制有：

- 锁 （重点） chapter15进行讨论
- 时间戳
- 多版本和快照隔离

T_1	T_2
$\text{read}(A)$ $A := A - 50$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$
$\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$B := B + \text{temp}$ $\text{write}(B)$ commit

图 14-5 调度 4：一个导致不一致状态的并发调度

• 何为可串行化的调度 如何判断是否可串行化

为确定一个调度是否冲突可串行化，我们这里给出了一个简单有效的 等价的一个串行调度方法。设 S 是一个调度，我们由 S 构造一个有向图，称为 **优先图** (precedence graph)。该图由两部分组成 $G = (V, E)$ ，其中 V 是顶点集， E 是边集，顶点集由所有参与调度的事务组成，边集由满足下列三个条件之一的边 $T_i \rightarrow T_j$ 组成：

- 在 T_j 执行 **read**(Q) 之前， T_i 执行 **write**(Q)。
- 在 T_j 执行 **write**(Q) 之前， T_i 执行 **read**(Q)。
- 在 T_j 执行 **write**(Q) 之前， T_i 执行 **write**(Q)。

如果优先图中存在边 $T_i \rightarrow T_j$ ，则在任何等价于 S 的串行调度 S' 中， T_i 必出现在 T_j 之前。

T_3	T_4
$\text{read}(Q)$	$\text{write}(Q)$
$\text{write}(Q)$	

图 14-9 调度 7



(3) 基于锁的协议

给数据项加锁的方式有多种，在这一节中，我们只考虑两种：

- 1. 共享的(shared)：如果事务 T_i 获得了数据项 Q 上的共享型锁 (shared-mode lock) (记为 S)，则 T_i 可读但不能写 Q 。
- 2. 排他的(exclusive)：如果事务 T_i 获得了数据项 Q 上的排他型锁 (exclusive-mode lock) (记为 X)，则 T_i 既可读又可写 Q 。

	S	X
S	true	false
X	false	false

图 15-1 锁相容性矩阵 comp

举一个例子，再考虑我们在第 14 章中介绍的银行业务系统。令 A 与 B 是事务 T_1 与 T_2 访问的两个账户，事务 T_1 从账户 B 转 50 美元到账户 A 上(见图 15-2)，事务 T_2 显示账户 A 与 B 上的总金额，即 $A + B$ (见图 15-3)。

```
T1: lock-X(B);
    read(B);
    B := B - 50;
    write(B);
    unlock(B);
    lock-X(A);
    read(A);
    A := A + 50;
    write(A);
    unlock(A).
```

图 15-2 事务 T_1

```
T2: lock-S(A);
    read(A);
    unlock(A);
    lock-S(B);
    read(B);
    unlock(B);
    display(A + B).
```

图 15-3 事务 T_2

在写数据之前加上排他锁还是会有问题，当前的事务未结束，就释放了B的锁，出现不一致的问题，因此考虑在事务结束之后再释放锁，但是会有死锁问题。锁的授予顺序很重要。

T_1	T_2	并发控制管理器
lock-x(B)		grant-x(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display(A + B)	
lock-x(A)		grant-x(A, T_1)
read(A)		
$A := A - 50$		
write(A)		
unlock(A)		

图 15-4 调度 1

遗憾的是，封锁可能导致一种不受欢迎的情形。考察如图 15-7 所示的有关事务 T_3 与 T_4 的部分调度，由于 T_3 在 B 上拥有排他锁，而 T_4 正在申请 B 上的共享锁，因此 T_4 等待 T_3 释放 B 上的锁；类似地，由于 T_4 在 A 上拥有共享锁，而 T_3 正在申请 A 上的排他锁，因此 T_3 等待 T_4 释放 A 上的锁。于是，我们进入了这样一种哪个事务都不能正常执行的状态，这种情形称为死锁 (deadlock)。当死锁发生时，系统必须回滚两个事务中的一个。一旦某个事务回滚，该事务锁住的数据项就被解锁，其他事务就可以访问这些数据项，继续自己的执行。15.2 节将再讨论死锁处理这个问题。

```

lock-S(A);
read(B);
display(A + B);
unlock(A);
unlock(B);

```

图 15-6 事务 T_4
(事务 T_2 延迟释放锁)

• 两阶段封锁协议

保证可串行性的一个协议是两阶段封锁协议 (two-phase locking protocol)。该协议要求每个事务分两个阶段提出加锁和解锁申请。

1. 增长阶段 (growing phase)：事务可以获得锁，但不能释放锁。
2. 缩减阶段 (shrinking phase)：事务可以释放锁，但不能获得新锁。

- 死锁预防
- 死锁检测与恢复

Example

Suppose there are two transactions as below:

T1: Read(B); Read(A); A=B+1; Write(A)

T2: Read(B); Read(A); B=A+1; Write(B)

(1) Suppose that $A=2, B=2$ initially, the execution of a certain concurrent schedule results in $A=3, B=3$, please tell whether the concurrent schedule is correct or not, Why?

解：T1、T2 的串行执行结果为 $A=3, B=4$, T2、T1 的串行执行结果为 $B=3, A=4$, T1、T3 并发执行的结果与任一串行执行结果均不相同，所以该调度不正确。

(2) Please give a (conflict) serializable schedule for T1 and T2 using lock-based protocol along with (以及) its execution result.

T1	T2
LOCK-S(B)	
READ(B) (=2)	
LOCK-X(A)	
READ(A) (=2)	
A=B+1	
UNLOCK(B)	
	LOCK-X(B) T1 已释放 B!
	READ(B) (=2)
Write(A) (=3)	
UNLOCK(A)	
	LOCK-S(A) T1 已释放 A!
	READ(A) (=3)
	B=A+1
	Write(B) (=4)
	UNLOCK(A)
	UNLOCK(B)

Question3 : ER图的设计与绘制

ER图包括实体集和联系集，实体集可以通过现实的抽象来获得，比如学生、课程等。关系集反映实体之间的联系，比如学生和课程之间的联系就是选课。

具体来说，一个实体集是由n多属性构成的，一个关系集是两个实体之间关联的集合，包括一对多、多对一、多对多。

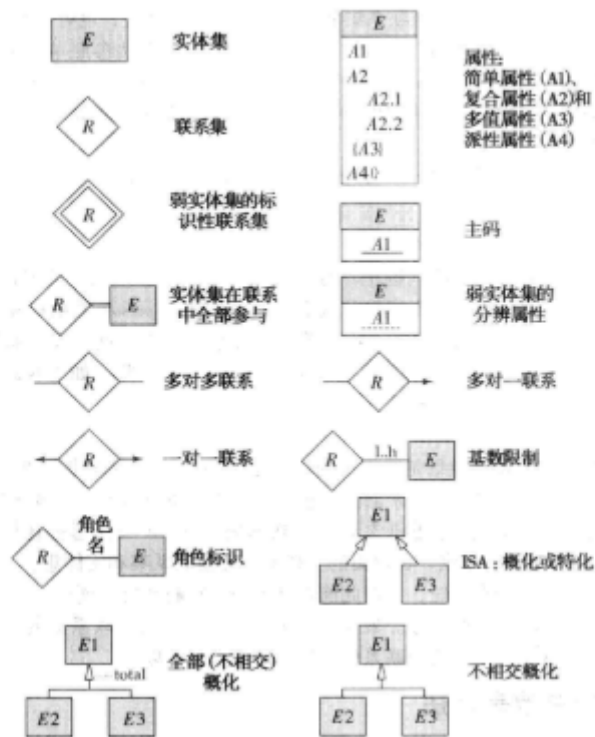


图 7-24 E-R 图表示法中使用的符号

实体集E包含
简单属性A1、
复合属性A2、
多值属性A3、
派生属性A4、
以及主码A1

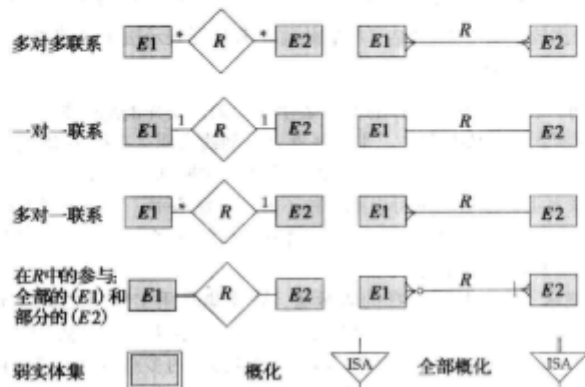


图 7-25 其他可选择的 E-R 图表示法

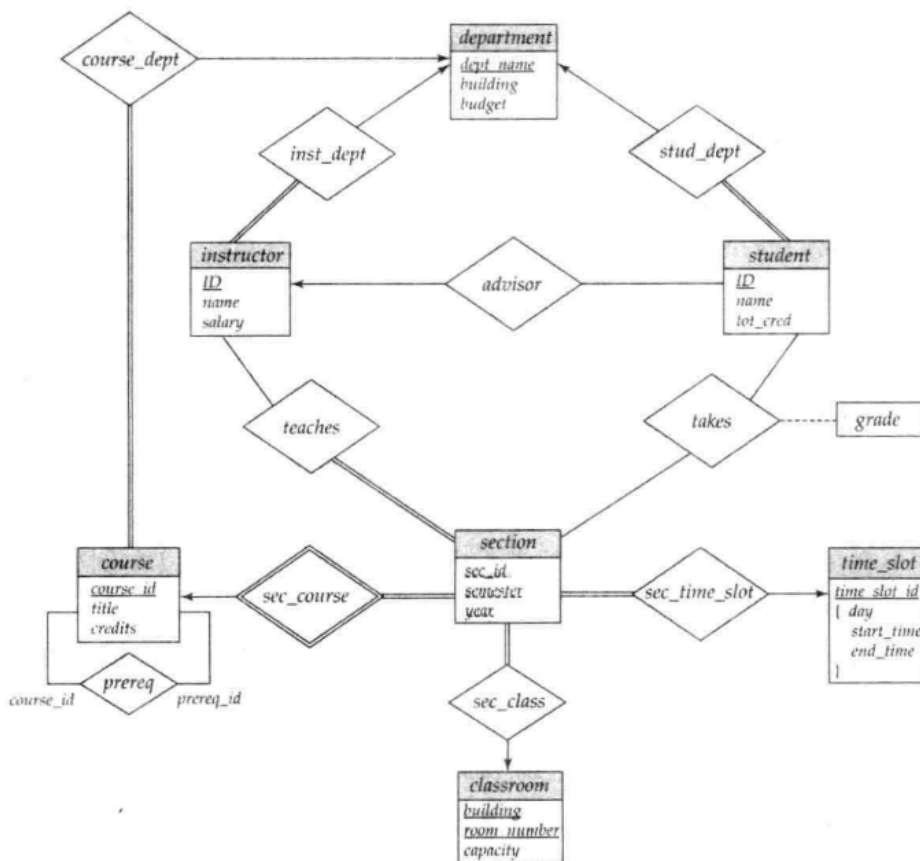


图 7-15 大学的 E-R 图

Example: 如何绘制ER图

何时使用弱实体集？当前实体缺少主键，或者说，主键依赖于另一个实体的时候。

如，虎溪校区的教室编号为1430，A区也有一个教室编号为1430，从教室中抽象出的属性没有可以成为主键的，因此需要使用校区这个实体的主键来进行区分，此时教室就成了一个依赖于校区的实体集。

题 8. 设某医院病房计算机管理中心需要如下的信息表：

科室（科名、科地址、科电话、医生姓名）

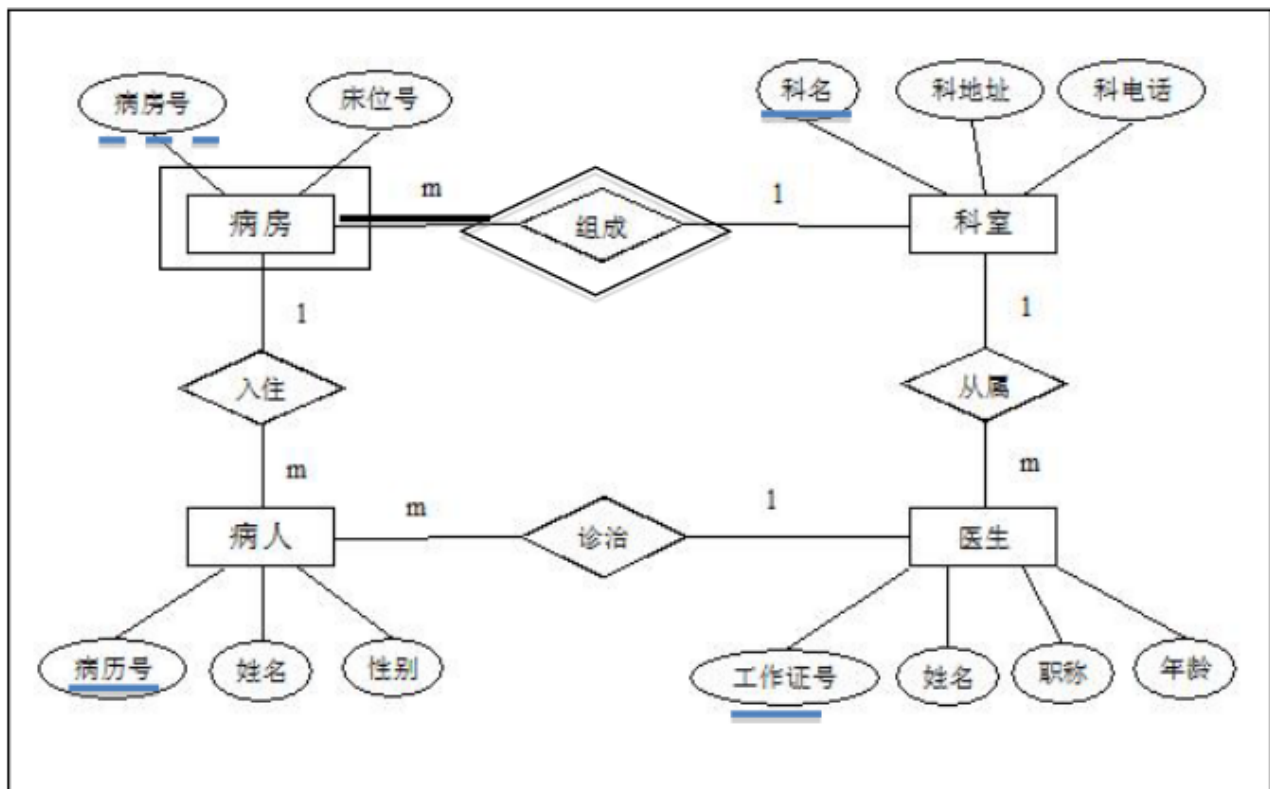
病房（病房号、床位号、所属科室名）

医生（姓名、职称、所属科室名、年龄、工作证号）

病人（病历号、姓名、性别、诊断、主管医生、病房号）

其中，一个科室有多个病房、多个医生，一个病房只能属于一个科室，一个医生只属于一个科室，一个医生可负责多个病人的诊治，但一个病人的主管医生只有一个。

请完成该系统的概念模型设计，并作出相应的 E-R 图（简述理由）。



解：参考答案如下：

(1) 本题的概念模型设计得到的 E-R 图应为：

注：其中的病房为弱实体集。（因各科室都有病房，分别独立编号）

(2) (设计理由) 其转换得到的关系模式结构如下：

科室（科名，科地址，科电话）

病房（病房号，床位号，科名）

医生（工作证号，姓名，职称，年龄，科名）

病人（病历号，姓名，性别，主管医生工作证号，病房号，科室）

而每个实体集的候选码、外码分别说明如下：

科室的候选码是‘科名’。

病房的候选码是‘科名’+‘病房号’，

外键是‘科名’，参照关系模式科室。

医生的候选码是‘工作证号’，

外键是‘科名’，参照关系模式科室。

病人的候选码是‘病历号’，

一个外键是‘主管医生工作证号’，参照关系模式医生；

另一外键是‘病房号’+‘科室’，参照关系模式病房。

Question4: ER图转关系模式

这里主要提 弱实体集的转化和联系集的转化

7.6.3 弱实体集表示

设 A 是具有属性 $a_1, a_2, \dots, a_m$ 的弱实体集, 设 B 是 A 所依赖的强实体集, 设 B 的主码包括属性 $b_1, b_2, \dots, b_n$ 。我们用名为 A 的关系模式表示实体集 A , 该模式的每个属性对应以下集合中的一个成员:

$$\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$$

对于从弱实体集转换而来的模式, 该模式的主码由其所依赖的强实体集的主码与弱实体集的分辨符组合而成。除了创建主码之外, 还要在关系 A 上建立外码约束, 该约束指明属性 $b_1, b_2, \dots, b_n$ 参照关系 B 的主码。外码约束保证表示弱实体的每个元组都有一个表示相应强实体的元组与之对应。

以图 7-15 所示 E-R 图中的弱实体集 *section* 为例说明。该实体集有属性: *sec_id*、*semester* 和 *year*。*section* 实体集所依赖的实体集 *course* 的主码是 *course_id*。因此, 用来表示 *section* 的模式具有下面的属性:

$$\text{section}(\underline{\text{course_id}}, \underline{\text{sec_id}}, \text{semester}, \text{year})$$

该主码由实体集 *course* 的主码和 *section* 的分辨符 (即 *sec_id*、*semester* 以及 *year*) 组成。我们还在模式 *section* 上建立了一个属性 *course_id* 参照 *course* 模式的主码的外码约束, 以及完整性约束“级联删除”^①。由于外码约束上的“级联删除”规范, 如果一个 *course* 实体被删除, 那么所有与它相关联的 *section* 实体也被删除。

7.6.4 联系集表示

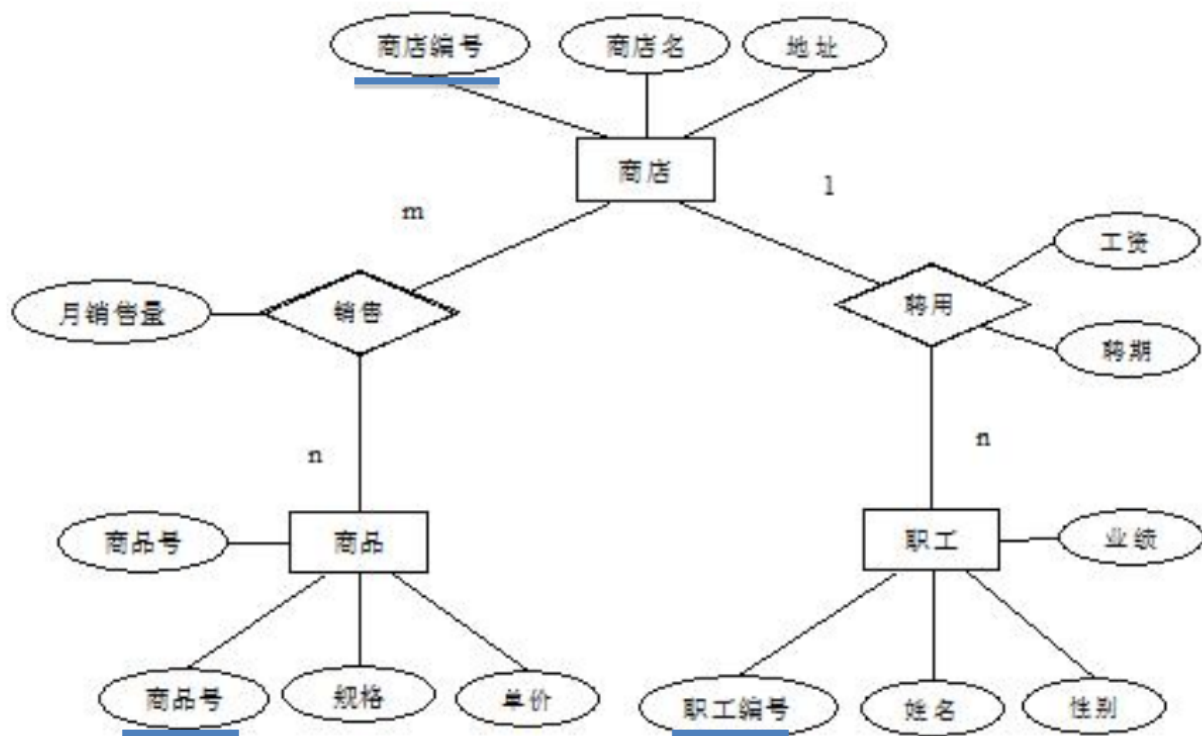
设 R 是联系集, 设 $a_1, a_2, \dots, a_m$ 表示所有参与 R 的实体集的主码的并集构成的属性集合, 设 R 的描述性属性 (如果有) 为 $b_1, b_2, \dots, b_n$ 。我们用名为 R 的关系模式表示该联系集, 下面集合中的每一项表示为模式的一个属性:

$$\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$$

7.3.3 节介绍过如何为一个二元联系集选择主码。如在该节中所看到的, 从所有相关实体集中取所有主码属性能够用来标识一个指定元组, 但是对于一对一、多对一和一对多联系集, 这会得到一个比我们所需要的主码大的属性集合。因而如下选择主码:

- 对于多对多的二元联系, 参与实体集的主码属性的并集成为主码。
- 对于一对一的二元联系集, 任何一个实体集的主码都可以选作主码。这个选择是任意的。
- 对于多对一或一对多的二元联系集, 联系集中“多”的那一方的实体集的主码构成主码。

题 10. 设某商业集团的数据库设计得到如下 E-R 图：



解：该 E-R 图转换得到的关系、主码和外码分如下：

商店（商店编号，商店名，地址）；

商店编号为主码。

职工（职工编号，姓名，性别，业绩，商店编号，聘期，工资）；

职工编号为主码，

商店编号为外码，参照的关系模式为商店。

商品（商品号，商品名，规格，单价）

商品号为主码。

销售（商店编号，商品号，月销售量）

商店编号+商品号为主码，

商店编号为一外码，参照的关系模式为商店；

商品号为另一外码，参照的关系模式为商品。

Question5: 一些有用的材料

- 数据库系统概念（原书第6版）
- 日常PPT

- <https://github.com/ScienceLi1125/CQU-Study>